

Reto 3 – API Gateway y Resiliencia Sincrónica

Contexto

Este reto continúa el desarrollo del **sistema de onboarding y offboarding de empleados**. En retos anteriores se construyeron:

- Un servicio básico de empleados con Docker (Reto 1)
- Orquestación con Docker Compose, persistencia y comunicación REST entre servicios (Reto 2)

El sistema funciona, pero tiene dos debilidades que se van a volver caras muy pronto:

1. No tiene un borde. Hoy el cliente habla directamente con cada microservicio en su propio puerto: `localhost:8080` para empleados, `localhost:8081` para departamentos. Cada servicio nuevo agrega un puerto más que el cliente debe conocer. Y cuando en el Reto 5 haya que exigir autenticación, no habrá un solo lugar donde ponerla: habrá que repetir la validación del token en cada uno de los servicios, en cada uno de los lenguajes.

2. Reintenta contra un servicio que ya sabe que está caído. En el Reto 2 se agregaron timeouts y reintentos. Eso protege contra un fallo momentáneo, pero si departamentos lleva diez minutos caído, empleados gasta diez minutos esperando timeouts y reintentando llamadas condenadas a fallar — consumiendo hilos, conexiones y memoria que necesita para atender lo que sí puede responder.

Este reto resuelve las dos: introduce el **API Gateway** como punto de entrada único, y el **Circuit Breaker** como la protección que faltaba en la llamada síncrona.

Objetivo

Establecer un **punto de entrada único** al ecosistema mediante un API Gateway, cerrando el acceso directo a los microservicios, e implementar un **Circuit Breaker** que evite que el fallo de una dependencia síncrona degrade a quien la consume.

Requisitos generales

- Implementar un **API Gateway** como servicio del `docker-compose.yml`.
- **Solo el Gateway expone puertos al host.** Ningún microservicio es accesible directamente.
- Todas las peticiones del cliente pasan por el Gateway, con rutas estables y documentadas.
- Implementar un **Circuit Breaker** en la comunicación empleados → departamentos, con estado observable y respuesta de *fallback*.
- Documentar la arquitectura y demostrar el comportamiento del Circuit Breaker ante un fallo real.

1. El API Gateway como punto de entrada único

1.1 Qué problema resuelve

Un API Gateway es un servicio que se sitúa **delante** de los microservicios y recibe todo el tráfico externo. Su valor no es el enrutamiento en sí (eso ya lo hacía Docker exponiendo puertos), sino tener **un solo lugar** donde poner todo lo que aplica a todas las peticiones:

Responsabilidad	Sin Gateway	Con Gateway
Punto de acceso	N puertos que el cliente debe conocer	Uno solo
Autenticación (Reto 5)	Repetida en cada servicio, en cada lenguaje	En un solo punto
Superficie expuesta	Todos los servicios alcanzables desde fuera	Solo el Gateway
Métricas del sistema (Reto 8)	Hay que agregarlas de N fuentes	El Gateway ve todo el tráfico
Cambiar un servicio de puerto o de host	Rompe a los clientes	Invisible desde fuera

Esta decisión es difícil de revertir. Si los retos siguientes se construyen sobre N puertos directos, para cuando llegue el Reto 6 la suite de pruebas y para cuando llegue el Reto 8 la configuración de Prometheus estarán atadas a esos puertos, y meter un gateway después implicará rehacerlas. Por eso se hace ahora, con solo dos servicios en juego.

1.2 Qué tipo de Gateway implementar

Existen dos familias, y la elección importa para lo que viene:

Tipo	Ejemplos	Fortaleza	Limitación
Enrutador declarativo	Traefik, Nginx	Configuración por labels, balanceo automático, cero código	No ejecuta lógica propia: no compone respuestas ni transforma cargas útiles sin plugins
Gateway de aplicación	Spring Cloud Gateway, Express, FastAPI, Go net/http	Es código: puede validar, transformar, agregar y componer	Hay que escribirlo y mantenerlo

Para este reto se pide un Gateway de aplicación (de código). La razón es la trayectoria del curso:

- En el **Reto 5** el Gateway debe validar el JWT y aplicar reglas de autorización.
- En el **Reto 10** debe validar tokens por **JWKS** y propagar la identidad del usuario a los servicios internos como cabeceras.
- El **proyecto final** contempla la composición de respuestas (combinar empleado + perfil).

Ninguna de las tres es natural en un enrutador declarativo. Traefik aparecerá en el **Reto 11**, donde sí brilla: como **balanceador de carga** delante de este Gateway, cuando haya varias instancias que balancear.

1.3 Opciones por lenguaje

Lenguaje	Opción típica
Java	Spring Cloud Gateway
Node.js	Express + http-proxy-middleware
Python	FastAPI + httpx
Go	net/http/httputil.ReverseProxy
.NET	YARP (Yet Another Reverse Proxy)

1.4 Requisitos del Gateway

Aspecto	Requisito
Puerto	Único puerto publicado al host (sugerido: 8080)
Enrutamiento	/empleados/* → empleados-service , /departamentos/* → departamentos-service
Puertos internos	Los microservicios usan <code>expose:</code> , nunca <code>ports:</code>
Propagación	Reenvía el cuerpo, las cabeceras y el código de estado sin alterarlos
Errores	Si el servicio destino no responde, retorna 503 con un cuerpo JSON descriptivo, no un error del framework
Salud propia	Expone su propio /health

1.5 El cambio en `docker-compose.yml`

Lo esencial es el paso de `ports:` a `expose:` en los servicios de negocio:

```

services:
  api-gateway:
    build: ./api-gateway
    ports:
      - "8080:8080"          # ← el ÚNICO puerto publicado al host
    environment:
      - EMPLEADOS_URL=http://empleados-service:8081
      - DEPARTAMENTOS_URL=http://departamentos-service:8082
    networks:
      - microservices-network

  empleados-service:
    build: ./empleados
    expose:
      - "8081"              # ← visible solo dentro de la red de Docker
    networks:
      - microservices-network

  departamentos-service:
    build: ./departamentos
    expose:
      - "8082"
    networks:
      - microservices-network

```

Verificación obligatoria: después del cambio, `curl http://localhost:8081/empleados` debe **fallar** (conexión rechazada) y `curl http://localhost:8080/empleados` debe funcionar.

Consecuencia inmediata: las URLs de todos los `curl` de los retos anteriores cambian. Actualice el README y su colección de Postman/Bruno para que apunten al Gateway. Desde este reto, **la URL base del sistema es una sola** — y así la esperan la suite de pruebas del Reto 6 y la configuración de Prometheus del Reto 8.

2. Circuit Breaker

2.1 El problema que resuelve

En el Reto 2 el servicio de empleados ya maneja timeouts y reintentos. Pero considere qué pasa cuando departamentos lleva caído varios minutos y llegan 100 peticiones de registro:

Sin Circuit Breaker:

$100 \text{ peticiones} \times (3 \text{ reintentos} \times 5\text{s de timeout}) = 1500 \text{ segundos de espera acumulada}$

→ hilos bloqueados, conexiones agotadas, empleados deja de responder

→ el fallo de departamentos se propagó a empleados: FALLO EN CASCADA

Con Circuit Breaker:

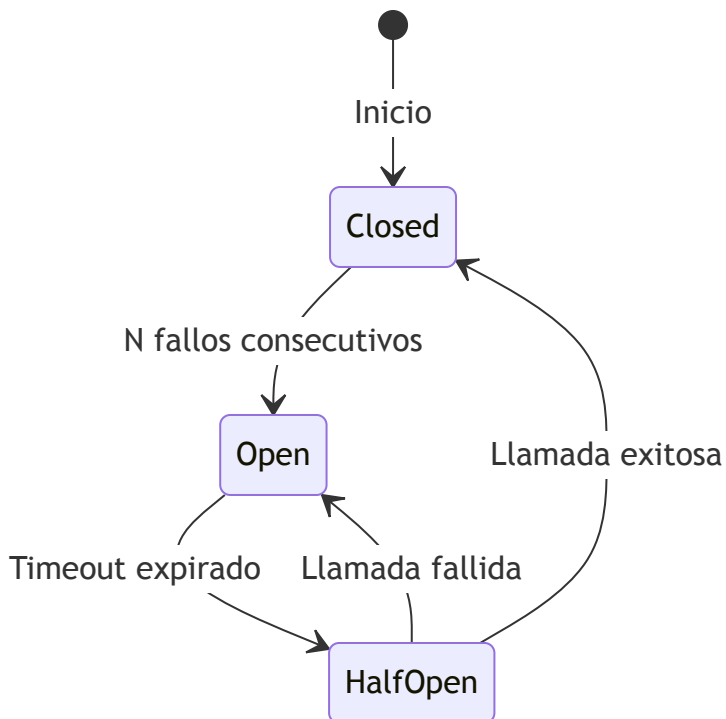
las primeras N peticiones fallan y abren el circuito

las 95 restantes reciben el fallback de inmediato, sin tocar la red

→ empleados sigue en pie, degradado pero vivo

La idea central: **cuando ya sabe que la dependencia está caída, dejar de intentarlo es lo correcto**. El Circuit Breaker es lo que convierte "reintentar" en "reintentar con criterio".

2.2 Estados



Estado	Comportamiento
CLOSED	Llamadas normales al servicio. Se cuentan los fallos.
OPEN	Llamadas bloqueadas. Respuesta inmediata del fallback, sin tocar la red .
HALF_OPEN	Se permite una llamada de prueba. Si funciona, cierra; si falla, vuelve a abrir.

El estado HALF_OPEN es el que hace que el sistema **se recupere solo**: nadie tiene que reiniciar nada cuando departamentos vuelve.

2.3 Configuración

Parámetro	Valor sugerido
Umbral de fallos	3-5 fallos consecutivos (o 50 % de una ventana de 10)
Timeout del circuito	30-60 segundos
Timeout de llamada	5 segundos (ya configurado en el Reto 2)

2.4 Lógica esperada

```
función registrarEmpleado(empleado):  
    si circuitBreaker.estaAbierto():  
        retornar fallback()  
  
    intentar:  
        respuesta = httpClient.get(departamentos + "/" + empleado.departamentoId)  
        .conTimeout(5 segundos)  
        circuitBreaker.registrarExito()  
        retornar procesarRegistro(empleado, respuesta)  
    capturar error:  
        circuitBreaker.registrarFallo()  
        si circuitBreaker.debeAbrir():  
            circuitBreaker.abrir()  
        retornar fallback()  
  
función fallback():  
    retornar {  
        estado: "PENDIENTE_VALIDACION",  
        mensaje: "Empleado registrado. Validación de departamento pendiente."  
    }  
}
```

2.5 Librerías por lenguaje

No implemente el patrón a mano: use la librería del ecosistema, que además expone métricas del estado del circuito (las va a necesitar en el Reto 11).

Lenguaje	Librería
Java	Resilience4j
Node.js	opossum
Python	pybreaker , tenacity
Go	sony/gobreaker , go-resiliency
.NET	Polly

2.6 El fallback es una decisión de negocio, no técnica

La parte difícil del Circuit Breaker no es abrirlo: es decidir **qué responder cuando está abierto**. Y esa respuesta depende del negocio, no del framework:

Opción	Consecuencia
Rechazar con 503	Nunca hay datos inconsistentes, pero RRHH no puede trabajar mientras departamentos esté caído
Registrar con estado: PENDIENTE_VALIDACION	RRHH sigue trabajando, pero queda un empleado con un departamento sin verificar que alguien deberá reconciliar
Aceptar con un departamento por defecto	Nunca haga esto: inventa datos

Elija una, justifíquela en el README, y explique cómo se reconcilia el estado si eligió la segunda. Esta pregunta —*¿prefiere disponibilidad o consistencia?*— es la misma que reaparecerá en el Reto 4 con la comunicación asíncrona y en el Reto 12 con el patrón Saga.

3. Pruebas del Sistema

3.1 Verificar el punto de entrada único

```
# A través del Gateway: debe funcionar
curl http://localhost:8080/departamentos
curl http://localhost:8080/empleados
```

```
# Acceso directo al servicio: DEBE FALLAR (conexión rechazada)
curl http://localhost:8081/departamentos
curl http://localhost:8082/empleados
```

```
# El Gateway responde a su propio health check
curl http://localhost:8080/health
```

3.2 Verificar el manejo de errores del Gateway

```
# Apagar un servicio y pedirle al Gateway que lo enrute
docker compose stop departamentos-service
curl -i http://localhost:8080/departamentos

# Debe retornar 503 con un cuerpo JSON descriptivo,
# no un stack trace ni una página de error del framework
docker compose start departamentos-service
```

3.3 Probar el Circuit Breaker

```
# 1. Crear un departamento con el sistema sano
curl -X POST http://localhost:8080/departamentos \
  -H "Content-Type: application/json" \
  -d '{"id": "IT", "nombre": "Tecnología"}'

# 2. Detener el servicio de departamentos
docker compose stop departamentos-service

# 3. Intentar registrar empleados y OBSERVAR EL TIEMPO DE RESPUESTA de cada uno
for i in {1..8}; do
  echo "--- Petición $i ---"
  time curl -s -X POST http://localhost:8080/empleados \
    -H "Content-Type: application/json" \
    -d '{"id": "E00'$i'", "nombre": "Test '$i'", "departamentoId": "IT"}'
done
```

Lo que debe observar —y es la evidencia central del reto:

- Las **primeras 3-5 peticiones** tardan segundos (timeout + reintentos): el circuito está **CLOSED**.
- A partir del umbral, las peticiones responden **casi instantáneamente** con el fallback: el circuito está **OPEN** y ya ni siquiera intenta la llamada.
- **Ese salto en el tiempo de respuesta es el Circuit Breaker funcionando.** Captúrelo.

4. Restaurar el servicio y esperar el timeout del circuito

```
docker compose start departamentos-service
```

```
sleep 35
```

5. Verificar la recuperación automática: sin reiniciar nada, vuelve a validar

```
curl -X POST http://localhost:8080/empleados \
```

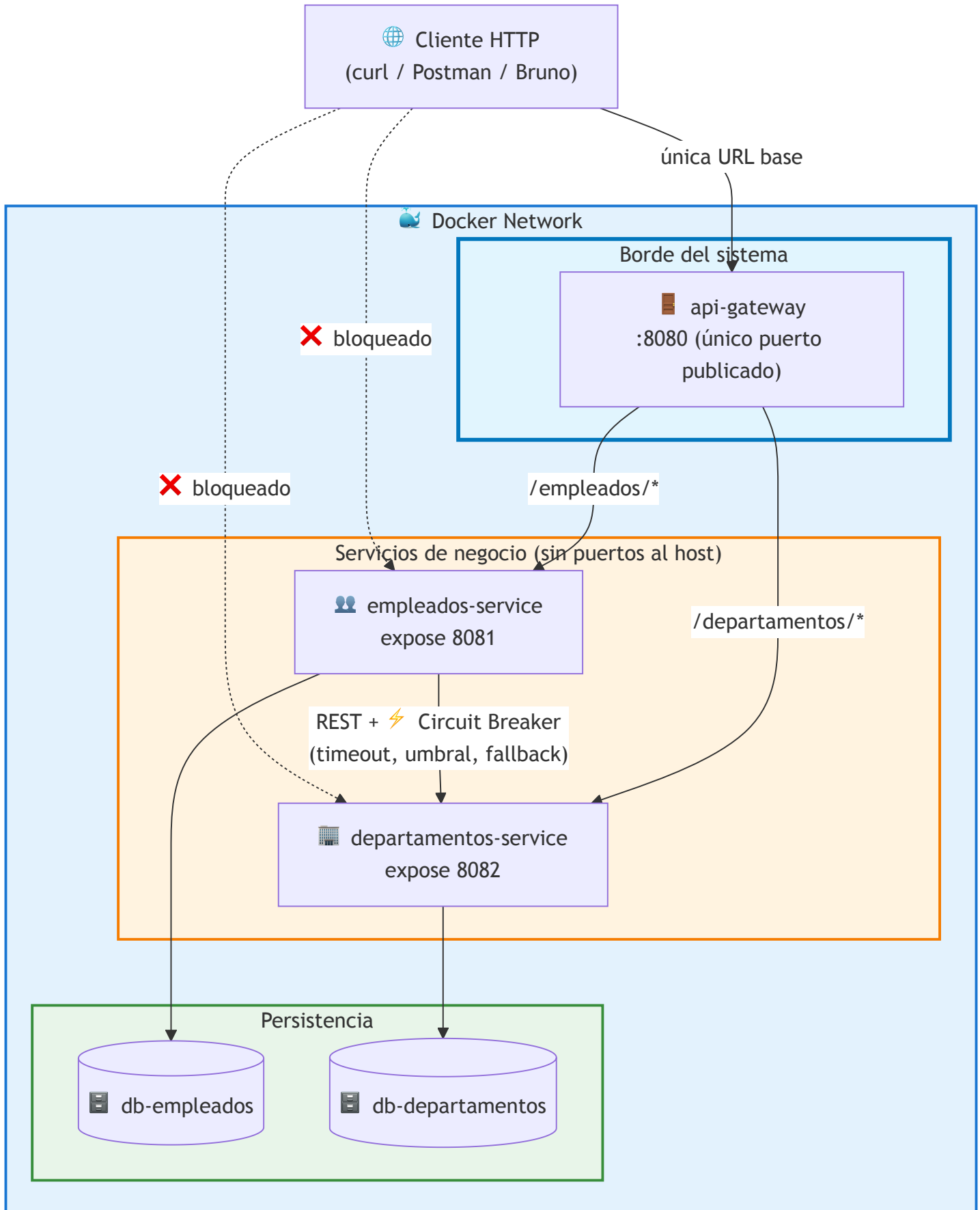
```
-H "Content-Type: application/json" \
```

```
-d '{"id": "E100", "nombre": "Recuperado", "departamentoId": "NO-EXISTE"}'
```

Debe responder 400 (departamento inexistente), lo que prueba que volvió a consultar d

El paso 5 usa un departamento **inexistente** a propósito: si respondiera con el fallback, no sabríamos si el circuito cerró o no. Un **400** solo puede venir de una consulta real a departamentos.

4. Diagrama de Arquitectura Esperada



Entregables

- **Código del API Gateway**, versionado como un microservicio más del ecosistema.
- `docker-compose.yml` actualizado: el Gateway con `ports:` , los servicios de negocio con `expose:` .
- **Código del Circuit Breaker** en el servicio de empleados.
- **README.md** con:
 - Justificación de la tecnología elegida para el Gateway.
 - Tabla de rutas del Gateway (ruta externa → servicio interno).
 - **La URL base del sistema**, que los retos siguientes van a usar.
 - Parámetros del Circuit Breaker elegidos y por qué.
 - **Justificación de la estrategia de fallback** y, si aplica, cómo se reconcilia el estado pendiente.
 - Instrucciones para reproducir la prueba del Circuit Breaker.
- **Evidencias** (capturas o video) de:
 - El acceso directo a un microservicio **rechazado**, y el mismo recurso funcionando por el Gateway.
 - El **salto en el tiempo de respuesta** al abrirse el circuito.
 - La recuperación automática tras restaurar el servicio.

Consideraciones

- El Gateway es un microservicio más: tiene su `Dockerfile` , su `/health` y, desde el Reto 8, su endpoint de métricas.
- No incluya lógica de negocio en el Gateway. Su trabajo es enrutar y, más adelante, aplicar preocupaciones transversales (autenticación, límites de tasa). Las reglas del dominio viven en los servicios.
- El Circuit Breaker va en **quien llama** (empleados), no en quien recibe. Es una protección para el consumidor, no para el proveedor.
- Los servicios deben mantener compatibilidad con el Reto 2: las rutas internas no cambian, solo deja de accederse a ellas desde fuera.

Lo que este reto habilita

Reto	Cómo se apoya en este
Reto 4 – Comunicación asíncrona	Los servicios nuevos entran al ecosistema detrás del Gateway
Reto 5 – Seguridad JWT	El Gateway es el punto único donde se valida el token
Reto 6 – Pruebas	La suite BDD apunta a una sola <code>BASE_URL</code> : la del Gateway
Reto 8 – Observabilidad	El Gateway es el primer objetivo de Prometheus: ve todo el tráfico del sistema
Reto 10 – SSO	El Gateway valida por JWKS y propaga la identidad a los servicios internos
Reto 11 – Escalabilidad	Traefik se coloca como balanceador y el Circuit Breaker se instrumenta en Grafana
Reto 13 – Gateway avanzado	Kong reemplaza o complementa este Gateway con límites de tasa y versionado

Criterios de Evaluación

El reto se evalúa sobre **5 puntos**, distribuidos de la siguiente forma:

#	Elemento	Valor	Aspectos a evaluar
1	API Gateway funcional	1.5	Gateway de aplicación implementado y desplegado en Compose. Enrutamiento correcto a ambos servicios, propagando cuerpo, cabeceras y códigos de estado. Manejo de error <code>503</code> con cuerpo JSON descriptivo cuando el destino no responde. <code>/health</code> propio.
2	Punto de entrada único	1.0	Solo el Gateway publica puertos al host; los servicios usan <code>expose: .</code> Evidencia de que el acceso directo a un microservicio es rechazado. README y colección de pruebas actualizados a la nueva URL base.

#	Elemento	Valor	Aspectos a evaluar
3	Circuit Breaker	1.5	Implementado con librería del ecosistema en la llamada empleados → departamentos. Los tres estados demostrables. Evidencia del salto en el tiempo de respuesta al abrirse y de la recuperación automática sin reinicio.
4	Estrategia de fallback	0.5	Fallback funcional y coherente. Decisión justificada en términos de negocio (disponibilidad vs. consistencia), con la reconciliación explicada si se eligió aceptar con validación pendiente.
5	Documentación y evidencias	0.5	README completo con tabla de rutas, parámetros del Circuit Breaker y capturas de las tres pruebas.

Nota: se evaluará especialmente la capacidad de **demostrar** el comportamiento ante fallos. Un Circuit Breaker que está en el código pero cuyo efecto no se puede evidenciar no cuenta como implementado.